- I accessed Claude through the Claude app and ChatGPT through its website in Google Chrome https://chatgpt.com/

Prompt:

Use two GenAI's to generate C++ code using objects, not functions, to implement the following requirements (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):

• Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.

• Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:

o Each digit from 1 to 9 must appear exactly once in each row.

o Each digit from 1 to 9 must appear exactly once in each column.

o Each digit from 1 to 9 must appear exactly once in each block.

• Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.

• The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.

• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.

• That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)

• Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).

• This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.

• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.

• Write a recursive object-oriented program to find the solution to the five puzzle files.

• The output for each solution, should include:

o The puzzle file name (e.g., puzzle1.txt)

o The puzzle from the file printed out.

o The solution to the puzzle printed out or if no solution is found, "No solution found" printed out.
• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
• To help you debug your program, solution1.txt is a solution to puzzle1.txt.

Overview:

Claude.cpp is the better program and is the program I chose to submit as my baseline program. Both programs found the same solutions for all five puzzles, but Claude.cpp is split into clear classes and has the required prologue and comments, while Chat.cpp has no comments at all.

Criteria	**Chat.cpp**	**Claude.cpp**	**Better?**
Correctness	Correct on all five puzzles (1 solution for puzzles 1-3, no solution for puzzle 4, 2 solutions for puzzle 5); rejects any file with extra text after the grid	Same results on all five puzzles; prints a specific error for a missing or short file	Tie
Execution time	Picks the empty cell with the fewest candidates: 61 recursive calls on puzzle 5. Worst case O(9^k) for k blanks	Fills cells in row order: 1,269 recursive calls on puzzle 5. Worst case O(9^k)	Chat
Space complexity	O(k) recursion depth; prints each solution as soon as it is found	O(k) recursion depth plus O(s) to store all s solutions before printing	Chat
Maintainability	134 lines, one class, no comments and no prologue	251 lines, three classes (Board, SudokuSolver, PuzzleRunner), prologue and comments on every function	Claude

Correctness

Case	**Chat.cpp**	**Claude.cpp**
puzzle1.txt	1 solution, matches solution1.txt	1 solution, matches solution1.txt
puzzle4.txt (no valid solution)	Prints "No solution found"	Prints "No solution found"

puzzle5.txt (two solutions)	Prints both solutions	Prints both solutions
File missing or fewer than 81 entries	"Could not read a valid 9x9 puzzle file."	Names the problem, e.g. "Could not open missing.txt" or "does not contain 81 entries"
Text after the 81 entries (solution1.txt)	Rejects the whole file	Reads the 81 entries and ignores the rest

Both programs give the same output for all five puzzles, so neither has the edge in correctness. The only differences are in how they report bad input files.

Execution time

Both programs are brute force depth-first search with backtracking, so the worst case is the same. Chat.cpp makes far fewer recursive calls because it always fills the most constrained cell first (61 calls versus 1,269 on puzzle 5). On these five puzzles both finish in about 2.5 ms, so the difference only matters on harder puzzles.

Space complexity

Both use O(k) stack depth for k blank cells. Claude.cpp also keeps every solution in a vector until printing, which is small here (at most two solutions).

Maintainability

Claude.cpp is easier to maintain. It is longer (251 lines versus 134) but each class has one job: Board holds and checks the grid, SudokuSolver does the recursion, and PuzzleRunner prints the report. Chat.cpp puts everything in one class with no comments or prologue.

Changes

The one change I made was to findEmpty in the Board class (lines 117-126). Instead of returning the first empty cell, it now returns the empty cell with the fewest legal digits, the same idea Chat.cpp uses. The output is identical, and the recursive calls on puzzle 5 dropped from 1,269 to 61.

```
    /*
     * Finds the empty cell with the fewest legal digits, so the search
     * branches as little as possible (a cell with 0 options ends the path fast).
     * Output: true and the cell's position, or false if the board is full.
     */
    bool findEmpty(int& row, int& col) const {
        int fewest = 10;
        for (int r = 0; r < SIZE; r++)
            for (int c = 0; c < SIZE; c++) {
                if (cells[r][c] != 0) continue;
                int options = 0;
                for (int d = 1; d <= 9; d++)
                    if (canPlace(r, c, d)) options++;
                if (options < fewest) {
```

```
                    fewest = options;
                    row = r;
                    col = c;
                    if (options <= 1) return true;
                }
            }
        return fewest < 10;
    }
```

Original Code

```
/*
 * Program: EECS 348 Assignment 4 - Recursive Sudoku Solver
 *
 * Description:
 *   C++ object-oriented program that solves Sudoku puzzles with a
 *   depth-first search with backtracking (brute force). For every puzzle
 *   file it prints the file name, the starting puzzle, and every solution
 *   found, or "No solution found" if the puzzle cannot be solved.
 *
 * Input:
 *   Puzzle file names on the command line. With no arguments the program
 *   reads puzzle1.txt through puzzle5.txt from the current folder.
 *   Each file has 9 rows of 9 entries separated by spaces; a digit 1-9 is a
 *   given value and an underscore (_) is a blank cell.
 *
 * Output:
 *   The file name, the puzzle as read, and each solution (or
 *   "No solution found") printed to the terminal.
 *
 * Collaborators: None
 * Other Sources: Claude (Anthropic)
 * Author: Joseph Akins
 * Creation Date: October 15, 2026
 * Revision Date: October 15, 2026
 *
 * Revisions:
 *   Initial object-oriented implementation.
 */

#include <fstream>
#include <iostream>
#include <string>
#include <vector>

using namespace std;

/*
 * Board
 *
 * Holds the 9x9 grid. 0 means an empty cell.
 * Knows how to load itself from a file, print itself,
 * and check whether a digit is allowed in a cell.
 */
class Board {
public:
    static const int SIZE = 9;

    Board() {
```

```
        for (int r = 0; r < SIZE; r++)
            for (int c = 0; c < SIZE; c++)
                cells[r][c] = 0;
    }

    /*
     * Reads 81 entries from the file.
     * Input: file name. Output: true if the file was read correctly,
     * false otherwise (error message stored in errorMessage).
     */
    bool load(const string& fileName) {
        ifstream in(fileName);
        if (!in) {
            errorMessage = "Could not open " + fileName;
            return false;
        }
        string token;
        int count = 0;
        while (in >> token && count < SIZE * SIZE) {
            int value;
            if (token == "_") {
                value = 0;
            } else if (token.size() == 1 && token[0] >= '1' && token[0] <= '9') {
                value = token[0] - '0';
            } else {
                errorMessage = "Invalid entry '" + token + "' in " + fileName;
                return false;
            }
            cells[count / SIZE][count % SIZE] = value;
            count++;
        }
        if (count != SIZE * SIZE) {
            errorMessage = fileName + " does not contain 81 entries";
            return false;
        }
        return true;
    }

    /*
     * Returns true if digit can go in (row, col) without repeating
     * in the same row, column, or 3x3 block. The cell itself is skipped.
     */
    bool canPlace(int row, int col, int digit) const {
        for (int i = 0; i < SIZE; i++) {
            if (i != col && cells[row][i] == digit) return false;
            if (i != row && cells[i][col] == digit) return false;
        }
        int blockRow = (row / 3) * 3;
        int blockCol = (col / 3) * 3;
        for (int r = blockRow; r < blockRow + 3; r++)
            for (int c = blockCol; c < blockCol + 3; c++)
                if ((r != row || c != col) && cells[r][c] == digit)
                    return false;
        return true;
    }

    /*
     * Checks that the given (filled) digits do not already break a rule.
     * A puzzle like that has no solution.
     */
```

```
    bool givensAreValid() const {
        for (int r = 0; r < SIZE; r++)
            for (int c = 0; c < SIZE; c++)
                if (cells[r][c] != 0 && !canPlace(r, c, cells[r][c]))
                    return false;
        return true;
    }

    /*
     * Finds the next empty cell (row by row).
     * Output: true and the cell's position, or false if the board is full.
     */
    bool findEmpty(int& row, int& col) const {
        for (row = 0; row < SIZE; row++)
            for (col = 0; col < SIZE; col++)
                if (cells[row][col] == 0) return true;
        return false;
    }

    int get(int row, int col) const { return cells[row][col]; }
    void set(int row, int col, int value) { cells[row][col] = value; }

    /* Prints the grid with blanks as _ and lines between the 3x3 blocks. */
    void print(ostream& out) const {
        for (int r = 0; r < SIZE; r++) {
            if (r > 0 && r % 3 == 0) out << "------+-------+------\n";
            for (int c = 0; c < SIZE; c++) {
                if (c > 0 && c % 3 == 0) out << "| ";
                if (cells[r][c] == 0) out << '_';
                else out << cells[r][c];
                if (c < SIZE - 1) out << ' ';
            }
            out << '\n';
        }
    }

    string errorMessage;

private:
    int cells[SIZE][SIZE];
};

/*
 * SudokuSolver
 *
 * Runs the recursive depth-first search with backtracking on a Board
 * and keeps every complete solution it finds.
 */
class SudokuSolver {
public:
    explicit SudokuSolver(const Board& start) : board(start) {}

    /* Searches the whole tree. Output: number of solutions found. */
    int solve() {
        solutions.clear();
        if (board.givensAreValid()) search();
        return static_cast<int>(solutions.size());
    }

    const vector<Board>& getSolutions() const { return solutions; }
```

```
private:
    Board board;
    vector<Board> solutions;

    /*
     * Recursive step: fill the next empty cell with each candidate digit
     * and recurse. A full board is a solution. If a cell has no candidates,
     * this path fails and the call returns so the caller can try its next digit.
     */
    void search() {
        int row, col;
        if (!board.findEmpty(row, col)) {   // all 81 cells filled
            solutions.push_back(board);
            return;
        }
        for (int digit = 1; digit <= 9; digit++) {
            if (board.canPlace(row, col, digit)) {
                board.set(row, col, digit);
                search();
                board.set(row, col, 0);      // backtrack
            }
        }
    }
};

/*
 * PuzzleRunner
 *
 * Loads one puzzle file, solves it, and prints the required report.
 */
class PuzzleRunner {
public:
    explicit PuzzleRunner(const string& name) : fileName(name) {}

    void run(ostream& out) const {
        out << "==============================\n";
        out << "Puzzle file: " << fileName << "\n";
        out << "==============================\n";

        Board puzzle;
        if (!puzzle.load(fileName)) {
            out << "Error: " << puzzle.errorMessage << "\n\n";
            return;
        }

        out << "Puzzle:\n";
        puzzle.print(out);
        out << '\n';

        SudokuSolver solver(puzzle);
        int count = solver.solve();
        if (count == 0) {
            out << "No solution found\n\n";
            return;
        }

        const vector<Board>& solutions = solver.getSolutions();
        for (int i = 0; i < count; i++) {
            if (count == 1) out << "Solution:\n";
```

```
            else out << "Solution " << (i + 1) << " of " << count << ":\n";
            solutions[i].print(out);
            out << '\n';
        }
    }

private:
    string fileName;
};

int main(int argc, char* argv[]) {
    vector<string> files;
    for (int i = 1; i < argc; i++) files.push_back(argv[i]);
    if (files.empty())
        for (int i = 1; i <= 5; i++)
            files.push_back("puzzle" + to_string(i) + ".txt");

    for (const string& f : files) {
        PuzzleRunner runner(f);
        runner.run(cout);
    }
    return 0;
}
```